

MarketNest

Multi-Vendor E-Commerce Platform

Architecture, Stack & Planning Document — v2.0

Updated: June 2026 · Confidential

Next.js 14	NestJS	Supabase	PostgreSQL	pgvector	Redis	Stripe
------------	--------	----------	------------	----------	-------	--------

This document is the single source of truth for the MarketNest platform. It covers the chosen technology stack, full system architecture, database design, security model, scalability strategy, CI/CD pipeline, AI roadmap, and development phasing plan. Every decision is made with 1 million+ concurrent users as the baseline requirement.

Table of Contents

1.	Executive Summary
2.	System Overview — Three Portals
3.	Product Ownership Model ★ NEW
4.	Seller Anonymity Model ★ NEW
5.	Technology Stack Decision
6.	Full Architecture Breakdown
7.	Database Design ★ UPDATED
8.	Authentication & Security Model
9.	Scalability & Performance Strategy
10.	Payment Integration
11.	AI / RAG Roadmap
12.	CI/CD Pipeline & DevOps
13.	Development Phases ★ UPDATED
14.	Environment Variables
15.	Folder Structure

1. Executive Summary

MarketNest is a scalable multi-vendor e-commerce platform built on a modern, cloud-native stack. It supports three distinct actor roles — Super Admin, Seller, and Buyer — each with a fully isolated portal and permission scope. The architecture is designed for high availability, sub-100ms page loads globally, and horizontal scalability beyond 1 million active users without architectural changes.

Version 2.0 additions:

- **Product Ownership Model** — three product types: Seller-owned, Platform-owned (MarketNest Official), and Seller-assigned by admin.
- **Seller Anonymity** — buyers never see seller identity, store names, or contact info. Isolation is enforced at API and UI levels.

Frontend	Next.js 14 (App Router) — SSR + ISR + Edge
Backend	NestJS on Node.js — typed, modular, DI-driven
Database	Supabase PostgreSQL — RLS, Realtime, managed
Vector DB	Supabase pgvector — AI-ready, no separate service
Cache	Redis via Upstash — sessions, cart, rate limiting
Hosting	Vercel (FE) + Railway/Render (API) — auto-scaling
Payments	Stripe + COD webhook — dual mode from launch
Auth	Supabase Auth — Google OAuth + email/password

2. System Overview — Three Portals

Super Admin Portal

- One account bootstrapped via seed script — no self-registration.
- Creates, suspends, and deletes seller accounts.
- Lists products directly under 'MarketNest Official' or on behalf of a seller.
- Views platform-wide analytics: GMV, orders, revenue, user counts.
- Manages product categories, banners, and featured listings.
- Overrides any order or payment dispute.
- Accesses full audit log of all system actions.
- Assigns product ownership (seller-owned / platform-owned / seller-assigned).

Seller Portal

- Account created by Super Admin only; seller receives email invite.
- Manages own product catalogue: add, edit, delete, stock control.
- Views own order dashboard: new, processing, shipped, complete.
- Reviews customer feedback and ratings for their products.

- Sees payout history and upcoming payment schedules.
- Cannot see other sellers' data (enforced by RLS at DB level).
- Identity is never exposed to buyers — anonymity enforced at API level.

Buyer Storefront

- Browses and adds to cart as a guest — no account required.
- Guest cart persisted in Redis with a session token (7-day TTL).
- Prompted to sign in or create account only at checkout.
- Google OAuth one-click sign-in or email/password registration.
- On first login, guest cart is merged into the new user account.
- Supports Cash on Delivery and online payment (Stripe) at checkout.
- Real-time order tracking after purchase.
- Never sees seller identity, store names, or seller contact information.

3. Product Ownership Model ★ NEW

MarketNest v2.0 introduces three product ownership types to support scenarios where vendors only want product listing without storefront visibility, and where the platform itself wants to sell directly. This is a core business decision that protects the platform's commission model.

Type	Owner	Listed Under	Who Fulfils	Buyer Sees
Seller-Owned	Seller	Seller (internal)	Seller	Product only — no seller name
Platform-Owned	Super Admin	MarketNest Official	Admin / 3PL	'MarketNest Official' badge
Seller-Assigned	Super Admin (on behalf of seller)	Seller (internal)	Seller	Product only — no seller name

3.1 How Admin Creates Products

- Admin navigates to **Products** → **Add Product** in the admin portal.
- A **Product Owner** dropdown appears with three options: *MarketNest Official* (platform-owned), or any active seller (seller-assigned).
- If *MarketNest Official* is selected, the product is stored with `owner_type = platform` and `seller_id = NULL`. It displays a 'MarketNest Official' badge on the storefront — no seller identity shown.
- If a seller is selected, the product is stored with `owner_type = seller_assigned` and the chosen `seller_id`. The seller can edit and fulfil but did not originally create it.
- Sellers create their own products with `owner_type = seller_owned`. This is the default flow for sellers.

3.2 Display Rules on Buyer Storefront

- All three product types render identically to buyers — same PDP layout, same cart flow.
- **Platform-owned** products show a small 'MarketNest Official' badge under the product title.
- **Seller-owned and Seller-assigned** products show no seller identifier whatsoever — only the product information.
- No seller name, store name, store link, or contact info ever appears on any buyer-facing page.

3.3 Fulfilment Logic

- Platform-owned: orders route to admin's fulfilment queue. Admin marks orders shipped manually or via 3PL webhook.
- Seller-owned / Seller-assigned: orders route to the assigned seller's order dashboard as normal. Payout calculated using the seller's `commission_rate`.
- Platform-owned orders have no payout calculation — revenue goes directly to platform.

4. Seller Anonymity Model ★ NEW

Hiding seller identity from buyers is standard practice for closed marketplaces (Amazon, Noon). It protects the platform's commission by preventing off-platform transactions and keeps the buying experience brand-consistent. This section defines every enforcement point.

4.1 Enforcement Layers

Database (RLS)	The sellers table has no SELECT policy for the buyer role. Buyers cannot query seller rows directly.
API (NestJS)	All product endpoints strip seller_id, seller email, store_name, store_slug, and store description before returning to buyers. A dedicated BuyerProductDTO contains only buyer-safe fields.
Frontend	No seller name, profile link, or contact info is rendered on any buyer-facing component. The seller_id field is never passed to the Next.js buyer app.
Search	Vector search and filter APIs for buyers never expose seller metadata in results.
Order emails	Order confirmation and status emails sent to buyers contain no seller identity — only order details and MarketNest branding.

4.2 What Buyers See vs What Internal Users See

Element	Buyer sees	Admin / Seller sees
Product page header	Product title + price	Product title + seller store name
Product metadata	Category, rating, description	+ seller_id, seller email
Order detail	Order ID, items, totals, tracking	+ seller name + seller contact
Shipping label	MarketNest branded	Seller name on internal docs only
Review prompt	Product name only	Includes seller_id for routing

4.3 MarketNest Official Entity

A virtual seller entity called **MarketNest Official** is created at system bootstrap (seed script). It has a dedicated seller row with store_slug = 'marketnest-official' and is_system = true. Products assigned to this entity display a subtle 'MarketNest Official' badge — the only seller-like information ever shown to buyers. This entity cannot be deleted, suspended, or modified by the admin UI.

5. Technology Stack Decision

5.1 Why Next.js 14 (App Router)

Server Components ship zero JS for static UI. Built-in ISR caches product pages at CDN, revalidated on update. Edge Middleware runs auth checks at CDN edge, not origin. For sub-100ms global loads this is the only sensible frontend choice. Vercel pairs perfectly with automatic image optimisation, analytics, and global CDN.

5.2 Why NestJS over direct Supabase calls

NestJS provides: (a) single enforced entry point for rate limiting and WAF, (b) domain-driven module separation, (c) testable RBAC Guards, (d) WebSocket gateways and queue workers in the same runtime, (e) **BuyerProductDTO serialisation** that strips all seller identity fields before response — this is the primary technical enforcement of seller anonymity.

5.3 Full Stack Summary

Layer	Technology	Reason
Frontend	Next.js 14 App Router	SSR/ISR/Edge, Vercel deploy
UI Library	Tailwind CSS + shadcn/ui	Utility-first, accessible components
Backend API	NestJS (Node.js 20)	Typed, modular, scalable
ORM	Prisma	Type-safe DB client, migrations
Primary DB	Supabase PostgreSQL 15	Managed, RLS, Realtime
Vector DB	Supabase pgvector	AI search, no extra cost
Cache	Redis (Upstash)	Sessions, cart, rate limits
File Storage	Supabase Storage	CDN-backed, signed URLs
Auth	Supabase Auth	Google OAuth, guest sessions
Queue	BullMQ + Redis	Async jobs, email workers
Payments	Stripe + COD webhook	Cards + cash on delivery
Email	SendGrid / Resend	Transactional email
SMS	Twilio	Order confirmations
Monitoring	Sentry + Datadog	Errors + APM
CI/CD	GitHub Actions + Vercel	Auto-deploy on push
Hosting FE	Vercel Pro	Edge, auto-scale
Hosting API	Railway / Render	Container, auto-scale

6. Full Architecture Breakdown

6.1 Client Layer

All three portals are Next.js applications in a Turborepo monorepo under separate subdomains: `admin.marketnest.com`, `sell.marketnest.com`, `marketnest.com`. Each uses React Server Components. Static product pages use ISR with 60-second revalidation — CDN serves cached HTML for 99% of traffic. The buyer app has no seller-referencing components at all; seller data types are excluded from the shared-types package for the buyer portal.

6.2 CDN / Edge Layer

Vercel global edge handles static assets, ISR pages, and Edge Middleware. Cloudflare WAF sits in front of Vercel and NestJS to absorb DDoS, apply geo-blocking, and run bot detection. This layer absorbs 90%+ of traffic — origin servers only receive dynamic, authenticated requests.

6.3 API Gateway — NestJS

Single entry point for all mutations and authenticated reads. Runs on Railway/Render behind a load balancer. Key responsibilities: JWT validation, RBAC Guard (SuperAdmin > Seller > Buyer > Guest), request rate limiting, input validation (class-validator), request logging, and Swagger docs at `/api/docs`. Critical addition: `ProductSerializerInterceptor` strips all seller identity fields from any response destined for a buyer or guest caller.

6.4 Service Modules

`AuthModule` — sign-up, sign-in, Google OAuth, guest tokens, cart-merge. `ProductModule` — CRUD with `owner_type` logic, `BuyerProductDTO` serialisation, image upload. `OrderModule` — cart management, checkout pipeline, order state machine, COD flow, platform fulfilment queue for platform-owned products. `PaymentModule` — Stripe `PaymentIntent`, webhook, payout (skipped for platform-owned orders). `NotificationModule` — email, SMS, push, Realtime (all MarketNest-branded, no seller identity). `AIModule` — embedding generation, RAG queries, recommendations.

6.5 Message Queue — BullMQ

Long-running tasks offloaded to BullMQ queues backed by Redis: `email-queue`, `inventory-queue`, `embedding-queue` (generate product embeddings on create/update), `export-queue` (CSV exports), `platform-fulfilment-queue` (admin notified of platform-owned orders needing dispatch).

6.6 Data Layer — Supabase

PostgreSQL is the primary store with RLS enforcing data isolation. Redis caches: user sessions (24h TTL), guest carts (7d TTL), product listings (60s TTL), rate-limit counters. Supabase Storage holds product images with CDN transform URLs. `pgvector` stores 1536-dim embeddings for semantic search and recommendations.

7. Database Design ★ UPDATED

All tables live in Supabase PostgreSQL with RLS enabled on every table. Supabase Auth manages the users table; application tables reference `auth.users(id)` via foreign key. Version 2.0 adds `owner_type` and `is_system` columns, and the MarketNest Official virtual seller row.

Table	Key Columns
profiles	id (FK auth.users), role (enum: superadmin seller buyer), full_name, avatar_url, phone, created_at
sellers	id, user_id (FK profiles), store_name, store_slug, description, is_active, is_system (bool — true for MarketNest Official), commission_rate, created_by (FK profiles)
categories	id, name, slug, parent_id (self-ref), image_url, sort_order, is_active
products	id, seller_id (nullable FK sellers — NULL for platform-owned), owner_type (enum: seller_owned platform_owned seller_assigned), category_id, title, description, price, compare_price, stock_qty, sku, images (jsonb), status (enum), embedding (vector 1536), created_at, updated_at
product_variants	id, product_id, name, options (jsonb), price_delta, stock_qty, sku
carts	id, user_id (nullable FK profiles), session_id (guests), created_at, updated_at
cart_items	id, cart_id, product_id, variant_id, quantity, unit_price, added_at
orders	id, buyer_id (FK profiles), status (enum), payment_method (cod online), subtotal, shipping_fee, total, shipping_address (jsonb), created_at, updated_at
order_items	id, order_id, product_id, seller_id (nullable — NULL for platform-owned), owner_type, quantity, unit_price, status
payments	id, order_id, stripe_payment_intent_id, amount, currency, status, paid_at
payouts	id, seller_id (NOT NULL — only seller-owned/assigned items generate payouts), amount, status, period_start, period_end, processed_at
reviews	id, product_id, buyer_id, rating (1-5), body, created_at
audit_logs	id, actor_id, action, entity_type, entity_id, metadata (jsonb), created_at

Key RLS Policies

- products — SELECT: everyone reads active products. INSERT/UPDATE/DELETE: `seller_id = auth.uid()` (seller-owned) OR `role = superadmin`.
- products — Buyers and guests NEVER receive `seller_id`, `store_name`, or any seller identity via RLS column-level security.
- orders — SELECT: `buyer_id = auth.uid()` OR seller has `order_items` with matching `seller_id` OR `role = superadmin`.
- sellers — SELECT: `role = superadmin` sees all; sellers see only their own row. Buyer role has NO select on this table.
- audit_logs — SELECT: superadmin only. INSERT: service role only (from NestJS).

- order_items — seller_id column masked for buyer-role reads via column-level security.

8. Authentication & Security Model

Supabase Auth	Handles all identity. JWT access tokens (15min TTL) + refresh tokens (30d). Stored in HTTP-only cookies, never localStorage.
Google OAuth	Buyer clicks 'Continue with Google'. Supabase redirects, creates profile with role=buyer automatically.
Guest session	NestJS issues a signed guest token (UUID v4) in a cookie. Cart stored in Redis keyed by this token. On login, cart merged.
Seller creation	Only super admin can call POST /admin/sellers. Email invite sent via SendGrid. Seller completes registration via invite link.
RBAC Guards	NestJS Guard reads JWT role claim. Routes decorated @Roles('superadmin' 'seller' 'buyer'). Mismatch → 403 immediately.
Serialisation Guard	ProductSerializerInterceptor on all buyer-facing product routes: strips seller_id, store_name, store_slug, seller_email before response.
Rate Limiting	100 req/min general, 10 req/min auth endpoints, 5 req/min checkout. Redis counters. Cloudflare WAF for DDoS shield.
Input Validation	All DTOs validated with class-validator. SQL injection impossible via Prisma parameterised queries. XSS prevented by React escaping.
Secrets	All secrets in Vercel/Railway env vars. Never committed to git. Rotated quarterly. Supabase service role key — NestJS only, never browser.

9. Scalability & Performance Strategy

Target: handle 1 million+ concurrent users without architectural changes.

CDN-first rendering	Next.js ISR means product, homepage, and category pages are pre-rendered on Vercel global CDN. Origin API never receives these. Handles 90%+ of buyer traffic.
Redis caching layers	Hot product catalogue in Redis (60s TTL). User sessions in Redis. Cart state in Redis. Page view counts flushed to DB every 5 minutes. Removes 95% of read load from PostgreSQL.
DB connection pooling	Supabase PgBouncer in transaction mode. NestJS pools 10 connections per pod. At 10 pods = 100 DB connections, within Supabase Pro limits.
Read replicas	Supabase Pro read replicas in different regions. Heavy reads (product search, order history) route to nearest replica. All writes to primary.
Horizontal API scaling	NestJS is stateless (state in Redis/Postgres). Railway/Render auto-scales pods on CPU/memory. Zero configuration required.
Async queue offloading	Any operation > 200ms moved to BullMQ. Emails, inventory, embedding generation, invoice PDFs, admin fulfilment notifications — all async.
Image optimisation	Supabase Storage CDN transform URL. Next.js Image component serves WebP with correct device dimensions. No full-size images in browser.
Database indexes	Critical: products(seller_id), products(category_id), products(owner_type), products(status), orders(buyer_id), order_items(seller_id), cart_items(cart_id).

10. Payment Integration

Two payment modes supported at launch. Online uses Stripe PaymentIntent API. Flow: (1) buyer reaches checkout, (2) NestJS creates PaymentIntent and returns client_secret, (3) Next.js renders Stripe Elements, (4) buyer completes payment, (5) Stripe webhook fires payment_intent.succeeded, (6) order moves to confirmed and fulfilment begins.

Cash on Delivery creates the order with status=pending_cod and triggers seller notification (or admin notification for platform-owned products). Delivery confirmation triggers payout calculation for seller-owned/assigned items only. Platform-owned items generate no payout record.

Stripe webhook events handled: payment_intent.succeeded, payment_intent.payment_failed, charge.dispute.created, account.updated (for Connect). Payouts to sellers are batched weekly via Stripe Connect or manual bank transfer, after deducting the commission_rate from the sellers table. Platform-owned order revenue flows directly to platform account.

11. AI / RAG Roadmap

Phase 1 — Semantic Search

Timeline: Launch

Embedding via OpenAI text-embedding-3-small (1536 dims) stored in pgvector. Buyer search uses cosine similarity. Finds 'blue cotton summer dress' even if product says 'azure lightweight frock'. All product types (seller-owned, platform-owned, seller-assigned) are indexed identically.

Phase 2 — Recommendations

Timeline: Month 3

BullMQ worker builds user preference vector from order history and browsing events. Nightly job computes top-20 recommended products per user via pgvector ANN, stored in recommendations table. Homepage 'For you' section reads this table.

Phase 3 — AI Shopping Assistant

Timeline: Month 6

Chat widget powered by Claude claude-sonnet-4-20250514 via Anthropic API with RAG over product catalogue. User types natural language, system embeds query, retrieves top-10 products via pgvector, Claude generates personalised recommendation. No seller identity in context sent to Claude.

Phase 4 — Seller AI Tools

Timeline: Month 9

AI-generated product description writer, automated pricing suggestions, demand forecasting using historical order data.

12. CI/CD Pipeline & DevOps

Repository: GitHub monorepo (apps/web, apps/api, packages/shared-types, packages/ui). Branch strategy: main (production), staging (QA), feature/* (development).

- Push to feature/* → GitHub Actions: lint, type-check, unit tests. PR required to merge.
- Merge to staging → Actions builds Docker image for API, deploys to Railway staging. Vercel auto-deploys Next.js to preview URL.
- Merge to main → Full test suite + integration + Playwright E2E. On pass: API to Railway production (rolling, zero downtime). Vercel auto-deploys frontend.
- Database migrations: Prisma migrate deploy runs automatically in API startup script. Always backward-compatible.
- Secrets: GitHub Actions secrets + Railway/Vercel env vaults. Never in code.
- Monitoring: Sentry for runtime errors. Datadog APM traces every HTTP request and DB query. Slack alerts for p95 latency > 500ms or error rate > 1%.

13. Development Phases ★ UPDATED

Phase	Timeline	Key Deliverables
Phase 1 — Foundation (Weeks 1–4)	Weeks 1–4	Monorepo (Turborepo), ESLint, TypeScript strict. Supabase schema + RLS + Auth. NestJS skeleton: AuthModule, RBAC guards. Next.js App Router: route groups admin/seller/buyer. Tailwind + shadcn/ui. CI/CD: GitHub Actions, Vercel, Railway. MarketNest Official seed entity.
Phase 2 — Core Commerce (Weeks 5–9)	Weeks 5–9	ProductModule with owner_type logic + BuyerProductDTO serialisation (seller anonymity). CartModule: Redis cart, merge on login. OrderModule: checkout pipeline, COD flow, platform-fulfilment queue. PaymentModule: Stripe + payout skip for platform-owned. Seller portal UI. Buyer storefront: homepage, category, PDP (no seller info), cart, checkout.
Phase 3 — Admin & Notifications (Weeks 10–13)	Weeks 10–13	Admin portal: seller management, product ownership UI (owner dropdown), platform analytics, category admin. NotificationModule: SendGrid, Twilio, Realtime. BullMQ queues. Review/rating system. Payout calculation (seller-owned/assigned only). Full audit logging.
Phase 4 — AI & Polish (Weeks 14–18)	Weeks 14–18	pgvector embeddings on all product types. Semantic search (cosine similarity). Performance audit: Lighthouse, Core Web Vitals. Security audit: OWASP Top 10, seller data leak pen-test. Load testing: k6 at 10k concurrent. Beta launch.

14. Environment Variables & Configuration

Variable	Purpose
NEXT_PUBLIC_SUPABASE_URL	Supabase project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY	Supabase anon key (browser-safe)
SUPABASE_SERVICE_ROLE_KEY	Supabase service role — NestJS only, never browser
DATABASE_URL	PostgreSQL pooled connection (port 6543)
DIRECT_URL	Direct connection (port 5432) for Prisma migrations
REDIS_URL	Upstash Redis URL
JWT_SECRET	NestJS JWT signing secret
STRIPE_SECRET_KEY	Stripe secret key — NestJS only
STRIPE_WEBHOOK_SECRET	Stripe webhook signing secret
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY	Stripe public key for Stripe Elements
SENDGRID_API_KEY	SendGrid transactional email
TWILIO_ACCOUNT_SID	Twilio SMS
TWILIO_AUTH_TOKEN	Twilio SMS
OPENAI_API_KEY	OpenAI embeddings (text-embedding-3-small)
ANTHROPIC_API_KEY	Claude API for AI assistant
GOOGLE_CLIENT_ID	Google OAuth client ID
GOOGLE_CLIENT_SECRET	Google OAuth client secret

15. Folder Structure

```
marketnest/  
├── apps/  
│   ├── web/ # Next.js 14  
│   │   ├── app/  
│   │   │   ├── (buyer)/ # Buyer storefront – NO seller types imported  
│   │   │   ├── (seller)/ # Seller portal  
│   │   │   └── (admin)/ # Super admin portal  
│   │   └── api/ # NestJS API  
│   └── src/  
│       ├── auth/  
│       ├── products/ # owner_type logic, BuyerProductDTO  
│       ├── orders/ # platform-fulfilment queue  
│       ├── payments/ # payout skip for platform-owned  
│       ├── notifications/ # MarketNest-branded only  
│       ├── ai/  
│       └── admin/
```

```
■■■ packages/
■   ■■■ shared-types/           # SellerDTO excluded from buyer barrel exports
■   ■■■ ui/
■   ■■■ utils/
■■■ supabase/
■   ■■■ migrations/
■   ■■■ seed.sql                # MarketNest Official entity bootstrapped here
■■■ .github/workflows/
■■■ turbo.json
■■■ package.json
```

End of Document — MarketNest Architecture & Planning v2.0